
TRÍ TUỆ NHÂN TẠO

Machine Learning

Phần 7 — Dimensionality Reduction

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	3
1 Giảm chiều dữ liệu (Dimensionality Reduction)	4
1.1 Hai hướng chính	4
2 Lựa chọn đặc trưng (Feature Selection)	5
2.1 Phương pháp lọc (Filter Methods)	5
2.2 Phương pháp bọc (Wrapper Methods)	5
2.3 Phương pháp nhúng (Embedded Methods)	6
2.4 Phân tích thành phần chính (PCA)	6
2.5 Loại trừ đặc trưng đệ quy (RFE)	7
2.6 Ví dụ — Breast Cancer Wisconsin	7
3 Trích xuất đặc trưng (Feature Extraction)	9
3.1 Ví dụ — Trích xuất đặc trưng	9
3.2 Ưu điểm của trích xuất đặc trưng	10
3.3 Nhược điểm của trích xuất đặc trưng	11
4 Loại dần ngược (Backward Elimination)	11
4.1 Triển khai trong Python	11
4.2 Ví dụ đầy đủ	13
5 Xây dựng đặc trưng tiến (Forward Feature Construction)	15
5.1 Các bước thuật toán	16
5.2 Ví dụ triển khai Python	16
6 Bộ lọc tương quan cao (High Correlation Filter)	18
6.1 Thuật toán	18
6.2 Ví dụ Python — dataset Diabetes	19
6.3 Ưu và nhược điểm	19
7 Bộ lọc phương sai thấp (Low Variance Filter)	20
7.1 Thuật toán	20
7.2 Ví dụ Python — dataset Diabetes	21
7.3 Ưu và nhược điểm	21
8 Tỷ lệ giá trị thiếu (Missing Values Ratio)	22
8.1 Thuật toán	22
8.2 Ví dụ Python — Diabetes (OpenML)	23
8.3 Ưu và nhược điểm	24
9 Phân tích thành phần chính (PCA)	24
9.1 Lý thuyết — các bước đầy đủ	25
9.2 Ví dụ Python — Iris	26
9.3 Ưu và nhược điểm	27

Lời nói đầu

Nguồn

Tài liệu biên soạn và dịch đầy đủ từ Tutorialspoint Machine Learning — mục Dimensionality Reduction: https://www.tutorialspoint.com/machine_learning/machine_learning_dimensionality_reduction.htm.

Cách dùng

- Đọc sau phần Clustering Algorithms.
- Chạy code từ thư mục 7.DimensionalityReduction/.

Đặng Tấn Quý

1 Giảm chiều dữ liệu (Dimensionality Reduction)

Giảm chiều dữ liệu

Trong học máy, giảm chiều là quá trình **giảm số đặc trưng (feature) hoặc biến** trong tập dữ liệu trong khi vẫn giữ được nhiều thông tin gốc nhất có thể. Nói cách khác, đây là cách **đơn giản hóa** dữ liệu bằng cách giảm độ phức tạp.

Khi nào cần giảm chiều?

Nhu cầu giảm chiều xuất hiện khi dataset có **rất nhiều** đặc trưng:

- Dễ **overfitting** và làm mô hình phức tạp hơn.
- Khó **trực quan hóa** dữ liệu.
- Làm **chậm** quá trình huấn luyện.

1.1 Hai hướng chính

Lựa chọn đặc trưng (Feature Selection)

Chọn một **tập con** các đặc trưng gốc theo tiêu chí như mức quan trọng hoặc mức liên quan đến biến mục tiêu.

Các kỹ thuật lựa chọn đặc trưng phổ biến

- **Filter Methods** — lọc theo thống kê, độc lập mô hình.
- **Wrapper Methods** — đánh giá hiệu năng mô hình khi thêm/bớt feature.
- **Embedded Methods** — chọn feature **gắn trong** quá trình huấn luyện (Lasso, Ridge, cây quyết định, ...).

Trích xuất đặc trưng (Feature Extraction)

Biến dữ liệu thô thành tập đặc trưng **có ý nghĩa** hơn cho mô hình: giảm chiều bằng cách chọn, kết hợp hoặc biến đổi feature để tạo tập mới hữu ích hơn.

Lợi ích tổng quát

Giảm chiều có thể cải thiện **độ chính xác và tốc độ** mô hình, giảm overfitting và đơn giản hóa trực quan hóa dữ liệu.

2 Lựa chọn đặc trưng (Feature Selection)

Lựa chọn đặc trưng

Lựa chọn đặc trưng là bước quan trọng trong học máy: chọn một **tập con** các đặc trưng có sẵn để cải thiện hiệu năng mô hình.

Các kỹ thuật phổ biến

Dưới đây là một số kỹ thuật lựa chọn đặc trưng thường dùng.

2.1 Phương pháp lọc (Filter Methods)

Ý tưởng Filter

Phương pháp này đánh giá mức **liên quan** của từng đặc trưng bằng cách tính một độ đo thống kê (ví dụ: tương quan, thông tin tương hỗ, kiểm định chi-square, ...) rồi **xếp hạng** đặc trưng theo điểm số. Các đặc trưng có điểm thấp sẽ bị loại khỏi mô hình.

Filter trong Python — SelectPercentile và chi2

Để triển khai phương pháp lọc trong Python, dùng `SelectKBest` hoặc `SelectPercentile` từ module `sklearn.feature_selection`. Ví dụ chọn 10% đặc trưng tốt nhất theo chi-square (phù hợp dữ liệu không âm, phân loại):

```
from sklearn.feature_selection import SelectPercentile, chi2

selector = SelectPercentile(chi2, percentile=10)
X_new = selector.fit_transform(X, y)
```

Chi-square và kiểu dữ liệu

`chi2` yêu cầu giá trị đặc trưng **không âm**. Với đặc trưng liên tục (như bộ Breast Cancer), nên dùng `f_classif` hoặc `mutual_info_classif` thay vì `chi2`.

2.2 Phương pháp bọc (Wrapper Methods)

Ý tưởng Wrapper

Phương pháp này đánh giá hiệu năng mô hình khi **thêm hoặc bớt** đặc trưng, rồi chọn tập con đặc trưng cho kết quả tốt nhất. Cách này **tốn tính toán** hơn phương pháp lọc nhưng thường **chính xác** hơn.

Wrapper trong Python — RFE

Triển khai wrapper trong Python bằng hàm RFE (Recursive Feature Elimination) trong `sklearn.feature_selection`:

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

estimator = LogisticRegression()
selector = RFE(estimator, n_features_to_select=5)
selector = selector.fit(X, y)
X_new = selector.transform(X)
```

2.3 Phương pháp nhúng (Embedded Methods)

Ý tưởng Embedded

Phương pháp này **gắn** lựa chọn đặc trưng vào ngay quá trình xây dựng mô hình, ví dụ hồi quy Lasso, Ridge hoặc cây quyết định. Các phương pháp này gán **trọng số** cho từng đặc trưng; đặc trưng có trọng số nhỏ sẽ bị loại.

Embedded trong Python — Lasso

Triển khai embedded trong Python bằng Lasso hoặc Ridge từ `sklearn.linear_model`:

```
import pandas as pd
from sklearn.linear_model import Lasso

lasso = Lasso(alpha=0.1)
lasso.fit(X, y)
coef = pd.Series(lasso.coef_, index=X.columns)
important_features = coef[coef != 0]
```

2.4 Phân tích thành phần chính (PCA)

PCA trong ngữ cảnh lựa chọn đặc trưng

PCA là phương pháp học **không giám sát**: biến đổi các đặc trưng gốc thành tập các thành phần chính **không tương quan**, giải thích **phương sai tối đa** trong dữ liệu. Số thành phần chính có thể chọn theo ngưỡng, từ đó **giảm chiều** tập dữ liệu.

PCA trong Python

Triển khai PCA trong Python bằng PCA từ `sklearn.decomposition`:

```
from sklearn.decomposition import PCA

pca = PCA(n_components=3)
```

```
X_new = pca.fit_transform(X)
```

2.5 Loại trừ đặc trưng đệ quy (RFE)

RFE — Recursive Feature Elimination

Phương pháp này **loại dần** các đặc trưng kém quan trọng nhất cho đến khi còn tập đặc trưng quan trọng nhất. RFE dựa trên mô hình; có thể tốn tính toán nhưng cho kết quả tốt trên dữ liệu **chiều cao**.

RFECV trong Python

Triển khai RFE kèm xác thực chéo bằng RFECV (Recursive Feature Elimination with Cross Validation):

```
from sklearn.feature_selection import RFECV
from sklearn.tree import DecisionTreeClassifier

estimator = DecisionTreeClassifier()
selector = RFECV(estimator, step=1, cv=5)
selector = selector.fit(X, y)
X_new = selector.transform(X)
```

Kết hợp các kỹ thuật

Các kỹ thuật lựa chọn đặc trưng trên có thể dùng **riêng lẻ** hoặc **kết hợp** để cải thiện hiệu năng mô hình học máy. Cần chọn kỹ thuật phù hợp theo **kích thước** tập dữ liệu, **bản chất** đặc trưng và **loại** mô hình đang dùng.

2.6 Ví dụ — Breast Cancer Wisconsin

Bối cảnh ví dụ

Ví dụ dưới đây triển khai ba cách: lựa chọn đặc trưng đơn biến (univariate), loại trừ đặc trưng đệ quy có xác thực chéo (RFECV), và phân tích thành phần chính (PCA).

Bộ dữ liệu Breast Cancer

Dùng bộ **Breast Cancer Wisconsin (Diagnostic)** có sẵn trong scikit-learn: 569 mẫu, 30 đặc trưng; nhiệm vụ phân loại khối u **ác tính** (malignant) hay **lành tính** (benign).

Mã Python đầy đủ

```
# Import thư viện và bộ dữ liệu
import numpy as np
from sklearn.datasets import load_breast_cancer
```

```

from sklearn.feature_selection import SelectKBest, f_classif, RFECV
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.decomposition import PCA

# Tai bo du lieu
data = load_breast_cancer()
X = data.data
y = data.target

# Loc don bien: f_classif (phu hop dac trung lien tuc)
selector = SelectKBest(f_classif, k=10)
X_new = selector.fit_transform(X, y)

# Chia tap huan luyen / kiem tra
X_train, X_test, y_train, y_test = train_test_split(
    X_new, y, test_size=0.3, random_state=42)

# Hoi quy logistic tren dac trung da chon
clf = LogisticRegression(max_iter=10000)
clf.fit(X_train, y_train)

# Danh gia tren tap kiem tra
accuracy = clf.score(X_test, y_test)
print("Accuracy using univariate feature selection: {:.2f}".format(accuracy)
    )

# RFECV
estimator = LogisticRegression(max_iter=10000)
selector = RFECV(estimator, step=1, cv=5)
selector.fit(X, y)
X_new = selector.transform(X)
scores = cross_val_score(LogisticRegression(max_iter=10000), X_new, y, cv=5)
print("Accuracy using RFECV feature selection: %0.2f (+/- %0.2f)" %
    (scores.mean(), scores.std() * 2))

# PCA
pca = PCA(n_components=5)
X_new = pca.fit_transform(X)
scores = cross_val_score(LogisticRegression(max_iter=10000), X_new, y, cv=5)
print("Accuracy using PCA feature selection: %0.2f (+/- %0.2f)" %
    (scores.mean(), scores.std() * 2))

```

Sua loi so voi ban goc Tutorialspoint

Ban goc dung chi2 va duong dan C:\... tren file diabetes.csv; day la **Breast Cancer** voi dac trung **lien tuc**, nen dung load_breast_cancer va f_classif. Them import cross_val_score va PCA con thieu trong ban goc.

Kết quả chạy (tham khảo)

Khi chạy mã trên, terminal có thể in tương tự:

```
Accuracy using univariate feature selection: 0.74
Accuracy using RFECV feature selection: 0.77 (+/- 0.03)
Accuracy using PCA feature selection: 0.75 (+/- 0.07)
```

So sánh nhanh ba cách trong ví dụ

- **Univariate + `f_classif`**: nhanh, độc lập mô hình; chọn $k = 10$ trong 30 đặc trưng.
- **RFECV**: tìm tập đặc trưng tốt theo hiệu năng mô hình và CV; tốn tính toán hơn.
- **PCA**: giảm chiều bằng biến đổi tuyến tính; không giữ tên đặc trưng gốc.

3 Trích xuất đặc trưng (Feature Extraction)

Trích xuất đặc trưng

Trích xuất đặc trưng thường dùng trong xử lý ảnh, nhận dạng giọng nói, xử lý ngôn ngữ tự nhiên và các ứng dụng khác, khi dữ liệu thô **chiều cao** và khó xử lý trực tiếp.

3.1 Ví dụ — Trích xuất đặc trưng

PCA trên bộ Iris

Dưới đây là ví dụ trích xuất đặc trưng bằng **Phân tích thành phần chính (PCA)** trên bộ Iris bằng Python.

Mã Python đầy đủ

```
# Import thư viện và bộ dữ liệu
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Tải bộ dữ liệu
iris = load_iris()

# Trích xuất đặc trưng bằng PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(iris.data)

# Trực quan hóa dữ liệu đã biến đổi
plt.figure(figsize=(7.5, 3.5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=iris.target)
plt.xlabel('PC1')
```

```
plt.ylabel('PC2')
plt.show()
```

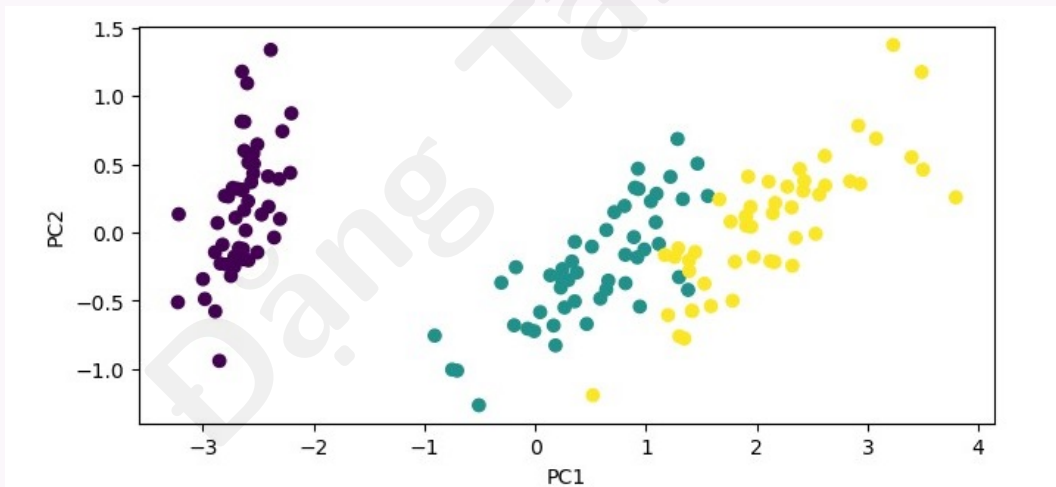
Giải thích từng bước trong mã

- Import các thư viện cần thiết: `sklearn` để PCA và `matplotlib` để vẽ.
- Nạp bộ Iris bằng `load_iris()`.
- Tạo `PCA()` với `n_components=2`: giảm chiều từ 4 đặc trưng xuống 2 thành phần chính.
- Biến đổi dữ liệu bằng `fit_transform()`, lưu kết quả vào `X_pca`.
- Vẽ bằng `plt.scatter()`, tô màu theo nhãn `iris.target`; trục PC1, PC2 là hai thành phần chính đầu tiên; hiển thị bằng `plt.show()`.

Kết quả đầu ra

Khi chạy chương trình, biểu đồ phân tán hai chiều của dữ liệu sau PCA sẽ hiển thị — các điểm thường tách theo ba loài hoa trong bộ Iris.

Hình minh họa kết quả



3.2 Ưu điểm của trích xuất đặc trưng

Ưu điểm

Dưới đây là các ưu điểm khi dùng trích xuất đặc trưng:

- **Giảm chiều (Reduced Dimensionality):** Trích xuất đặc trưng giảm chiều dữ liệu đầu vào bằng cách biến đổi thành tập đặc trưng mới. Dữ liệu dễ **trực quan hóa, xử lý và phân tích** hơn.
- **Cải thiện hiệu năng (Improved Performance):** Trích xuất đặc trưng có thể

cải thiện hiệu năng thuật toán học máy nhờ tạo tập đặc trưng **có ý nghĩa hơn**, nắm bắt thông tin cốt lõi từ dữ liệu gốc.

- **Lựa chọn đặc trưng (Feature Selection)**: Trích xuất đặc trưng có thể dùng để **lựa chọn** tập con đặc trưng liên quan nhất, hữu ích nhất cho mô hình.
- **Giảm nhiễu (Noise Reduction)**: Trích xuất đặc trưng giúp **giảm nhiễu** bằng cách lọc đặc trưng không liên quan hoặc kết hợp các đặc trưng có liên quan.

3.3 Nhược điểm của trích xuất đặc trưng

Nhược điểm

Dưới đây là các nhược điểm khi dùng trích xuất đặc trưng:

- **Mất thông tin (Loss of Information)**: Trích xuất đặc trưng có thể **mất thông tin** vì phải giảm chiều. Dữ liệu biến đổi có thể không chứa hết thông tin gốc; một phần thông tin bị mất trong quá trình biến đổi.
- **Overfitting**: Trích xuất đặc trưng có thể dẫn tới **overfitting** nếu đặc trưng biến đổi quá phức tạp hoặc số đặc trưng chọn quá lớn.
- **Độ phức tạp (Complexity)**: Trích xuất đặc trưng có thể **tốn tính toán** và thời gian, đặc biệt với tập dữ liệu lớn hoặc kỹ thuật phức tạp như học sâu (deep learning).
- **Chuyên môn miền (Domain Expertise)**: Trích xuất đặc trưng đòi hỏi **hiểu biết miền** để chọn và biến đổi đặc trưng hiệu quả; cần nắm dữ liệu và bài toán để chọn đúng đặc trưng mang thông tin cho mô hình.

4 Loại dần ngược (Backward Elimination)

Backward Elimination

Loại dần ngược là kỹ thuật **lựa chọn đặc trưng** trong học máy nhằm chọn các đặc trưng **quan trọng nhất** cho mô hình dự đoán. Ban đầu xét **tất cả** đặc trưng, sau đó **lặp loại bỏ** đặc trưng kém ý nghĩa nhất cho đến khi có tập con tốt nhất về hiệu năng.

4.1 Triển khai trong Python

Các bước triển khai

1. Import các thư viện: `pandas`, `numpy`, `statsmodels.api`.
2. Nạp tập dữ liệu vào `DataFrame` (ví dụ bộ Pima Indians Diabetes).
3. Định nghĩa biến dự đoán `X` và biến mục tiêu `y`.
4. Thêm cột hằng số 1 vào `X` để biểu diễn **hệ số chặn** (intercept).

5. Dùng **OLS** (Ordinary Least Squares) từ `statsmodels` để khớp hồi quy tuyến tính đa biến với mọi biến dự đoán.
6. Xem **p-value** của từng biến dự đoán, loại biến có **p-value cao nhất** (ít ý nghĩa nhất).
7. Lặp bước 5 và 6 cho đến khi mọi biến còn lại có p-value **dưới mức ý nghĩa** (ví dụ 0,05).

Vòng lặp p-value

Mỗi lần lặp, `regressor_OLS.summary()` liệt kê p-value từng hệ số. Biến có p-value **lớn nhất** (thường $> 0,05$) được coi là **không có ý nghĩa thống kê** với biến phụ thuộc trong mô hình hiện tại; loại biến đó rồi ước lượng lại OLS trên tập còn lại. Quá trình dừng khi **tất cả** p-value còn lại nhỏ hơn ngưỡng (ví dụ 0,05) — đó là tập đặc trưng tối ưu theo tiêu chí này.

Bước 1 — Import

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
```

Bước 2 — Nạp dữ liệu diabetes

```
from sklearn.datasets import fetch_openml

diabetes = fetch_openml('diabetes', version=0, as_frame=True, parser='auto')
X = diabetes.iloc[:, :-1].values
y = diabetes.iloc[:, -1].astype(int).values
```

Bước 3–4 — Intercept

```
X = np.append(arr=np.ones((len(X), 1)).astype(int),
               values=X, axis=1)
```

Bước 5 — OLS với tất cả biến (ví dụ 6 biến đầu)

```
X_opt = X[:, [0, 1, 2, 3, 4, 5]]
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
regressor_OLS.summary()
```

Bước 6–7 — Loại dần theo p-value

```
X_opt = X[:, [0, 1, 3, 4, 5]]
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
```

```
regressor_OLS.summary()

X_opt = X[:, [0, 3, 4, 5]]
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
regressor_OLS.summary()

X_opt = X[:, [0, 3, 5]]
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
regressor_OLS.summary()

X_opt = X[:, [0, 3]]
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
regressor_OLS.summary()
```

Tập đặc trưng cuối

Tập biến dự đoán cuối cùng có p-value dưới mức ý nghĩa là tập đặc trưng **tối ưu** (theo tiêu chí loại dần OLS + p-value) cho mô hình.

4.2 Ví dụ đầy đủ

Triển khai hoàn chỉnh

Dưới đây là triển khai đầy đủ loại dần ngược trong Python (8 đặc trưng Pima + intercept, lặp loại theo summary):

Mã Python đầy đủ

```
# Import thu vien
import pandas as pd
import numpy as np
import statsmodels.api as sm
from sklearn.datasets import fetch_openml

# Tai bo diabetes (Pima Indians, OpenML)
diabetes = fetch_openml('diabetes', version=0, as_frame=True, parser='auto')

# Dinh nghĩa X và y
X = diabetes.iloc[:, :-1].values
y = diabetes.iloc[:, -1].astype(int).values

# Cột 1 cho intercept
X = np.append(arr=np.ones((len(X), 1)).astype(int),
              values=X, axis=1)

# OLS với tất cả 8 biến dự đoán (chỉ số 0 là intercept)
X_opt = X[:, [0, 1, 2, 3, 4, 5, 6, 7, 8]]
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
```

```
regressor_OLS.summary()

# Lap: loại biến p-value cao nhất, ước lượng lại
X_opt = X[:, [0, 1, 2, 3, 5, 6, 7, 8]]
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
regressor_OLS.summary()

X_opt = X[:, [0, 1, 3, 5, 6, 7, 8]]
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
regressor_OLS.summary()

X_opt = X[:, [0, 1, 3, 5, 7, 8]]
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
regressor_OLS.summary()

X_opt = X[:, [0, 1, 3, 5, 7]]
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
regressor_OLS.summary()
```

Chỉ số cột trong X_opt

Cột 0 trong X sau khi thêm intercept là hằng số 1; các chỉ số 1..8 tương ứng 8 đặc trưng gốc (Pregnancies, Glucose, ...). Mỗi lần loại một chỉ số khỏi X_opt tương ứng loại một biến có p-value cao nhất trong `summary()` vừa in.

Kết quả đầu ra

Khi chạy chương trình, các bảng tóm tắt OLS lần lượt hiển thị; hình minh họa điển hình từ Tutorialspoint:

Hình minh họa

```

=====
                        OLS Regression Results
=====
Dep. Variable:          y      R-squared:          0.112
Model:                  OLS    Adj. R-squared:       0.106
Method:                 Least Squares    F-statistic:      19.22
Date:                  Thu, 27 Apr 2023    Prob (F-statistic): 4.81e-18
Time:                  17:17:34    Log-Likelihood:    -473.94
No. Observations:      767    AIC:              959.9
Df Residuals:          761    BIC:              987.7
Df Model:              5
Covariance Type:       nonrobust
=====
               coef    std err          t      P>|t|      [0.025    0.975]
-----
const         -0.0693     0.072     -0.960     0.337     -0.211     0.072
x1              0.0214     0.006      3.707     0.000      0.010     0.033
x2             -0.0003     0.001     -0.316     0.752     -0.002     0.001
x3              0.0005     0.000      3.497     0.000      0.000     0.001
x4              0.2171     0.050      4.334     0.000      0.119     0.315
x5              0.0064     0.002      3.770     0.000      0.003     0.010
=====
Omnibus:              521.181    Durbin-Watson:       2.006
Prob(Omnibus):         0.000    Jarque-Bera (JB):     82.038
Skew:                  0.538    Prob(JB):             1.53e-18
Kurtosis:              1.813    Cond. No.             678.
=====

```

Ưu và hạn chế

- **Ưu:** Dễ hiểu, gắn với kiểm định ý nghĩa thống kê (p-value); phù hợp hồi quy tuyến tính.
- **Hạn chế:** Giả định tuyến tính; tốn thời gian khi số đặc trưng rất lớn; thứ tự loại phụ thuộc mô hình hiện tại (không đảm bảo tập tối ưu toàn cục).

5 Xây dựng đặc trưng tiến (Forward Feature Construction)

Forward Feature Construction

Xây dựng đặc trưng tiến là phương pháp **lựa chọn đặc trưng**: bắt đầu từ tập đặc trưng **rỗng**, rồi **lập thêm** đặc trưng cho hiệu năng tốt nhất ở mỗi bước cho đến khi đạt số đặc trưng mong muốn.

Mục tiêu lựa chọn đặc trưng

Mục tiêu là xác định các đặc trưng **quan trọng nhất** để dự đoán biến mục tiêu, đồng thời bỏ qua đặc trưng ít quan trọng — vốn thêm **nhiều** vào mô hình và có thể gây **overfitting**.

5.1 Các bước thuật toán

Các bước Forward Feature Construction

1. **Khởi tạo** tập đặc trưng rỗng.
2. **Đặt** số đặc trưng tối đa cần chọn.
3. **Lặp** cho đến khi đủ số đặc trưng:
 - Với mỗi đặc trưng **chưa** nằm trong tập đã chọn: khớp mô hình gồm tập đã chọn **cộng** đặc trưng hiện tại, đánh giá hiệu năng trên tập **kiểm tra** (validation/test).
 - **Chọn** đặc trưng cho hiệu năng tốt nhất và **thêm** vào tập đã chọn.
4. **Trả về** tập đặc trưng đã chọn làm tập tối ưu cho mô hình.

Ưu điểm

Ưu điểm chính: **hiệu quả tính toán**, có thể áp dụng cho dữ liệu **chiều cao**.

Hạn chế

Không luôn dẫn tới tập đặc trưng **tối ưu toàn cục**, đặc biệt khi có đặc trưng **tương quan cao** hoặc quan hệ **phi tuyến** giữa đặc trưng và biến mục tiêu.

5.2 Ví dụ triển khai Python

Bối cảnh

Ví dụ dưới dùng hồi quy tuyến tính (LinearRegression), điểm **score** trên tập kiểm tra làm tiêu chí chọn đặc trưng tiến.

Mã Python đầy đủ

```
# Import thu vien
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.datasets import fetch_openml

# Tai bo diabetes
diabetes = fetch_openml('diabetes', version=0, as_frame=True, parser='auto')
X = diabetes.iloc[:, :-1].values
y = diabetes.iloc[:, -1].astype(int).values

# Chia tap huan luyen / kiem tra
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)
```



```
# Tập đặc trưng đã chọn (rỗng)
selected_features = set()

# Số đặc trưng tối đa
max_features = 8

# Lặp đến khi đủ 8 đặc trưng
while len(selected_features) < max_features:

    best_feature = None
    best_score = 0

    for i in range(X_train.shape[1]):

        if i in selected_features:
            continue

        X_train_selected = X_train[:, list(selected_features) + [i]]
        regressor = LinearRegression()
        regressor.fit(X_train_selected, y_train)

        X_test_selected = X_test[:, list(selected_features) + [i]]
        score = regressor.score(X_test_selected, y_test)

        if score > best_score:
            best_feature = i
            best_score = score

    selected_features.add(best_feature)

print('Selected Features:', list(selected_features))
print('Score:', best_score)
```

Cách đọc vòng lặp

Ở mỗi vòng `while`, thuật toán thử **từng** cột chưa chọn: huấn luyện trên `X_train` gồm các cột đã chọn + cột thử `i`, tính `regressor.score` trên `X_test` tương ứng. Cột có `score` cao nhất được **cố định** vào `selected_features`. In ra tập chỉ số và điểm sau mỗi bước.

Kết quả chạy (mẫu)

Khi chạy, terminal có thể in:

```
Selected Features: [1]
Score: 0.23530716168783583
Selected Features: [0, 1]
Score: 0.2923143573608237
Selected Features: [0, 1, 5]
Score: 0.3164103491569179
```

```

Selected Features: [0, 1, 5, 6]
Score: 0.3287368302427327
Selected Features: [0, 1, 2, 5, 6]
Score: 0.334586804842275
Selected Features: [0, 1, 2, 3, 5, 6]
Score: 0.3356264736550455
Selected Features: [0, 1, 2, 3, 4, 5, 6]
Score: 0.3313166516703744
Selected Features: [0, 1, 2, 3, 4, 5, 6, 7]
Score: 0.32230203252064216

```

Chỉ số đặc trưng

Các số 0, 1, 2, ... là **chỉ số cột** trong ma trận X (thứ tự cột của bộ Pima sau `fetch_openml`), không phải tên biến. Thứ tự thêm phụ thuộc dữ liệu và `random_state` khi chia train/test.

6 Bộ lọc tương quan cao (High Correlation Filter)

Bộ lọc tương quan cao

Kỹ thuật **lựa chọn đặc trưng** (filter method) dùng để phát hiện và loại các feature **tương quan mạnh** với nhau. Mục tiêu: giảm số biến dự báo, hạn chế **đa cộng tuyến** (multicollinearity) — khi hai hay nhiều biến giải thích gần như cùng một thông tin.

Nguyên tắc hoạt động

Tính hệ số tương quan giữa từng **cặp** feature; nếu $|r|$ vượt ngưỡng, loại **một** trong hai feature của cặp. Feature còn lại dùng huấn luyện mô hình.

6.1 Thuật toán

Các bước triển khai

1. Tính **ma trận tương quan** (correlation matrix) trên toàn bộ feature.
2. Đặt **ngưỡng** cho hệ số tương quan (ví dụ 0,8).
3. Tìm các **cặp feature** có $|r|$ lớn hơn ngưỡng.
4. Với mỗi cặp tương quan cao, **xóa một** feature (thường giữ feature có tương quan mạnh hơn với biến mục tiêu, hoặc xóa theo quy tắc cố định như trong ví dụ).
5. Huấn luyện mô hình trên tập feature còn lại.

6.2 Ví dụ Python — dataset Diabetes

Dữ liệu

Dùng bộ diabetes từ OpenML qua `sklearn.datasets.fetch_openml`; ngưỡng tương quan 0,8.

Mã đầy đủ

```
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_openml

data = fetch_openml("diabetes", version=1, as_frame=True, parser="auto").
    frame
X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

corr_matrix = np.corrcoef(X, rowvar=False)
threshold = 0.8

high_corr_indices = np.where(np.abs(corr_matrix) > threshold)
features_to_remove = set()

for i, j in zip(*high_corr_indices):
    if i != j and (j, i) not in features_to_remove:
        features_to_remove.add((i, j))

features_to_remove = list(features_to_remove)
X_filtered = np.delete(X, [j for i, j in features_to_remove], axis=1)

print('Shape of the filtered dataset:', X_filtered.shape)
```

Kết quả đầu ra

Output:

```
Shape of the filtered dataset: (768, 8)
```

Với ngưỡng 0,8, không có cặp feature nào vượt ngưỡng trên bộ diabetes chuẩn hóa từ OpenML, nên **giữ nguyên 8** biến dự báo và 768 mẫu.

6.3 Ưu và nhược điểm

Ưu điểm

- **Giảm đa cộng tuyến:** loại biến trùng lặp thông tin, ổn định hơn cho hồi quy tuyến tính.
- **Cải thiện hiệu năng mô hình:** ít feature $\Rightarrow$ huấn luyện nhanh hơn, ít nhiễu.

- **Đơn giản hóa mô hình:** dễ giải thích khi số biến giảm.
- **Tiết kiệm tài nguyên tính toán:** ma trận thiết kế nhỏ hơn.

Nhược điểm

- **Mất thông tin:** feature bị xóa có thể chứa tín hiệu hữu ích riêng.
- **Giả định tuyến tính:** chỉ đo tương quan Pearson; quan hệ phi tuyến có thể bị bỏ sót.
- **Ảnh hưởng biến phụ thuộc:** nếu hai feature tương quan cao đều quan trọng với y , xóa một có thể làm giảm chất lượng dự báo.
- **Selection bias:** quy tắc “xóa feature thứ hai trong cặp” có thể không tối ưu cho mọi bài toán.
- **Không đảm bảo tập tối ưu:** filter độc lập mô hình, có thể không khớp metric cuối (AUC, F1, ...).

7 Bộ lọc phương sai thấp (Low Variance Filter)

Bộ lọc phương sai thấp

Kỹ thuật **lựa chọn đặc trưng** loại các feature có **phương sai (variance)** nhỏ hơn ngưỡng. Feature gần như **không đổi** giữa các mẫu thường ít khả năng phân biệt được lớp mục tiêu.

Ý nghĩa

Phương sai đo mức **phân tán** của một biến. Biến hằng (hoặc gần hằng) mang ít “năng lực phân loại”; loại bỏ chúng có thể giảm nhiễu và chi phí tính toán mà không làm mất nhiều tín hiệu dự báo.

7.1 Thuật toán

Các bước triển khai

1. Tính **phương sai** từng feature trên toàn tập X .
2. Đặt **ngưỡng** phương sai (ví dụ 0,1).
3. Xác định chỉ số các feature có $\text{Var} < \text{ngưỡng}$.
4. **Xóa** các feature đó khỏi ma trận dữ liệu.
5. Dùng tập feature còn lại để huấn luyện mô hình.

7.2 Ví dụ Python — dataset Diabetes

Dữ liệu và ngưỡng

Bộ diabetes qua fetch_openml; ngưỡng phương sai 0,1.

Mã đầy đủ

```
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_openml

data = fetch_openml("diabetes", version=1, as_frame=True, parser="auto").
    frame
X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

variances = np.var(X, axis=0)
threshold = 0.1

low_var_indices = np.where(variances < threshold)
X_filtered = np.delete(X, low_var_indices, axis=1)

print('Shape of the filtered dataset:', X_filtered.shape)
```

Kết quả đầu ra

Output:

```
Shape of the filtered dataset: (768, 8)
```

Mọi feature trong diabetes OpenML đều có phương sai $\geq 0,1$, nên **không** feature nào bị loại.

7.3 Ưu và nhược điểm

Ưu điểm

- **Giảm overfitting:** bỏ biến gần như không đóng góp vào dự đoán.
- **Tiết kiệm tài nguyên:** ít cột $\Rightarrow$ huấn luyện và suy luận nhanh hơn.
- **Cải thiện hiệu năng:** giảm nhiễu từ biến “đứng yên”.
- **Đơn giản hóa mô hình:** tập feature gọn, dễ đọc.
- **Triển khai đơn giản:** chỉ cần thống kê mô tả, không cần nhãn y .

Nhược điểm

- **Mất thông tin:** biến phương sai thấp vẫn có thể quan trọng khi kết hợp phi tuyến với biến khác.
- **Giả định tuyến tính / độc lập:** không xét tương tác feature; loại một biến có thể phá cấu trúc ẩn.
- **Ảnh hưởng biến phụ thuộc:** biến ít biến thiên nhưng có quan hệ mạnh với y (ví dụ mã hiểm) có thể bị xóa nhầm.
- **Selection bias:** ngưỡng cố định có thể không phù hợp thang đo (cần chuẩn hóa trước khi lọc).
- **Nhạy thang đo:** feature chưa chuẩn hóa có thể bị đánh giá sai mức phương sai tương đối.

8 Tỷ lệ giá trị thiếu (Missing Values Ratio)

Tỷ lệ giá trị thiếu

Kỹ thuật **lựa chọn đặc trưng** loại các feature có **tỷ lệ phần trăm giá trị thiếu (NaN)** vượt ngưỡng. Feature “rỗng” quá nhiều khó học có ích và dễ gây **thiên lệch** khi impute hoặc bỏ mẫu không đồng nhất.

Nguyên tắc

Với mỗi cột, tính $\text{missing_ratio} = \frac{\# \text{NaN}}{\# \text{mẫu}}$. Nếu tỷ lệ $>$ ngưỡng (ví dụ 50%), **xóa cột** đó trước khi huấn luyện.

8.1 Thuật toán

Các bước triển khai

1. Nạp X dưới dạng mảng NumPy (hoặc DataFrame) có thể chứa `np.nan`.
2. Tính **tỷ lệ thiếu** theo từng cột: `np.isnan(X).mean(axis=0)`.
3. Đặt **ngưỡng** (ví dụ 0,5 tức 50%).
4. Thu thập chỉ số cột có tỷ lệ thiếu **lớn hơn** ngưỡng.
5. **Xóa** các cột đó; giữ y và các cột còn lại cho mô hình.

8.2 Ví dụ Python — Diabetes (OpenML)

Dữ liệu OpenML không có giá trị thiếu

Bản `fetch_openml("diabetes")` thường **không chứa NaN**. Đoạn dưới minh họa đúng **luồng tính toán**; kết quả shape thường giữ **8 feature**. Ví dụ Pima Indians gốc (CSV có ô trống) sau khi đọc và đặt NaN mới có thể loại cột — tùy cách tiền xử lý.

Mã đầy đủ

```
import numpy as np
from sklearn.datasets import fetch_openml

data = fetch_openml("diabetes", version=1, as_frame=True, parser="auto").
    frame
X = data.iloc[:, :-1].values.astype(float)
y = data.iloc[:, -1].values

missing_percentages = np.isnan(X).mean(axis=0)
threshold = 0.5

high_missing_indices = [
    i for i, pct in enumerate(missing_percentages) if pct > threshold
]

X_filtered = np.delete(X, high_missing_indices, axis=1)

print('Shape of the filtered dataset:', X_filtered.shape)
```

Kết quả đầu ra (diabetes OpenML)

Output:

```
Shape of the filtered dataset: (768, 8)
```

Không cột nào vượt ngưỡng 50% thiếu $\Rightarrow$ **giữ đủ 8 feature**, 768 mẫu.

Minh họa khi cần NaN tổng hợp

Để thấy bộ lọc “cắt” cột, có thể **chèn NaN giả lập** (chỉ cho thí nghiệm):

```
import numpy as np
from sklearn.datasets import fetch_openml

data = fetch_openml("diabetes", version=1, as_frame=True, parser="auto").
    frame
X = data.iloc[:, :-1].values.astype(float)
y = data.iloc[:, -1].values

rng = np.random.default_rng(0)
mask = rng.random(X.shape) < 0.6
```

```
X_demo = X.copy()
X_demo[mask] = np.nan

missing_percentages = np.isnan(X_demo).mean(axis=0)
threshold = 0.5
high_missing_indices = [
    i for i, pct in enumerate(missing_percentages) if pct > threshold
]
X_filtered = np.delete(X_demo, high_missing_indices, axis=1)
print('Shape after filter (demo):', X_filtered.shape)
```

8.3 Ưu và nhược điểm

Ưu điểm

- **Tiết kiệm tài nguyên:** bỏ cột khó xử lý, giảm chi phí impute và huấn luyện.
- **Cải thiện hiệu năng:** mô hình không phụ thuộc vào cột gần như trống.
- **Đơn giản hóa pipeline:** ít quy tắc điền khuyết phức tạp trên cột “chết”.
- **Giảm thiên lệch:** hạn chế cột thiếu không đồng nhất giữa các nhóm mẫu.

Nhược điểm

- **Mất thông tin:** cột thiếu nhiều vẫn có thể mang tín hiệu (ví dụ “không đo được” có ý nghĩa).
- **Ảnh hưởng dữ liệu còn lại:** xóa cột có thể làm mất biến liên quan chéo với cột khác.
- **Ảnh hưởng biến phụ thuộc:** cột thiếu nhưng dự báo y tốt có thể bị loại nhầm.
- **Selection bias:** ngưỡng cố định không xét mô hình cuối.
- **Phụ thuộc cách đọc dữ liệu:** CSV Pima cần quy ước mã hóa ô trống thành NaN; nếu đọc sai, tỷ lệ thiếu và kết quả lọc sẽ khác.

9 Phân tích thành phần chính (PCA)

PCA (Principal Component Analysis)

Kỹ thuật **không giám sát** giảm chiều: biến dữ liệu nhiều biến thành không gian chiều thấp hơn bằng các **thành phần chính** (principal components, PC) — tổ hợp tuyến tính của biến gốc, sắp xếp theo lượng **phương sai** được giữ lại.

Vai trò

PCA tìm cấu trúc và mối quan hệ giữa biến; ứng dụng trong nén dữ liệu, xử lý ảnh, trực quan hóa và tiền xử lý trước các thuật toán học máy. PC đầu giải thích nhiều biến thiên nhất, PC sau giải thích phần còn lại.

9.1 Lý thuyết — các bước đầy đủ**Bước 1: Chuẩn hóa dữ liệu**

PCA nhạy thang đo. Chuẩn hóa mỗi feature về **trung bình 0, phương sai 1**:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j},$$

với μ_j, σ_j là trung bình và độ lệch chuẩn cột j trên tập huấn luyện.

Bước 2: Ma trận hiệp phương sai

Với dữ liệu đã chuẩn hóa (mỗi cột đã căn giữa), ma trận hiệp phương sai:

$$\mathbf{C} = \frac{1}{n-1} \mathbf{Z}^\top \mathbf{Z},$$

trong đó $\mathbf{Z}$ là ma trận $n \times p$ (n mẫu, p feature). $\mathbf{C}$ đối xứng, mô tả biến thiên và **đồng biến** giữa các feature.

Bước 3: Trị riêng và vector riêng

Giải bài toán trị riêng:

$$\mathbf{C} \mathbf{v}_k = \lambda_k \mathbf{v}_k,$$

với $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$. Vector riêng $\mathbf{v}_k$ (đã chuẩn hóa) là hướng của **thành phần chính thứ k** ; λ_k là phương sai dọc theo hướng đó.

Bước 4: Chọn số thành phần

Giữ k PC đầu sao cho tổng phương sai giải thích đạt ngưỡng (ví dụ 95%), hoặc cố định $k = 2$ để vẽ đồ thị. Tỷ lệ phương sai giải thích của PC k :

$$\text{EVR}_k = \frac{\lambda_k}{\sum_{j=1}^p \lambda_j}.$$

Bước 5: Chiếu dữ liệu

Ma trận loadings (các vector riêng làm cột) $\mathbf{W}_k = [\mathbf{v}_1, \dots, \mathbf{v}_k]$. Tọa độ mới:

$$\mathbf{T} = \mathbf{Z} \mathbf{W}_k.$$

Mỗi hàng của $\mathbf{T}$ là điểm trong không gian k chiều dùng cho huấn luyện hoặc trực quan

hóa.

Hệ trục giao và giảm tương quan

Các PC mới **không tương quan** với nhau (trong giả thiết tuyến tính và sau chuẩn hóa). Đây là lý do PCA thường được dùng để giảm **đa cộng tuyến** trước hồi quy hoặc phân cụm.

Giả định và hạn chế lý thuyết

PCA là mô hình **tuyến tính**; outlier làm lệch **C**; PC khó diễn giải trực tiếp như tên feature gốc. Luôn chuẩn hóa (hoặc căn tỷ lệ hợp lý) trước khi áp dụng.

9.2 Ví dụ Python — Iris

Dữ liệu

Bộ iris qua `sklearn.datasets.load_iris`; chuẩn hóa thủ công, PCA 2 chiều, vẽ scatter theo nhãn loài.

Mã đầy đủ

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

iris = load_iris()
X = iris.data
y = iris.target

X_standardized = (X - np.mean(X, axis=0)) / np.std(X, axis=0)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_standardized)

print('Explained variance ratio:', pca.explained_variance_ratio_)

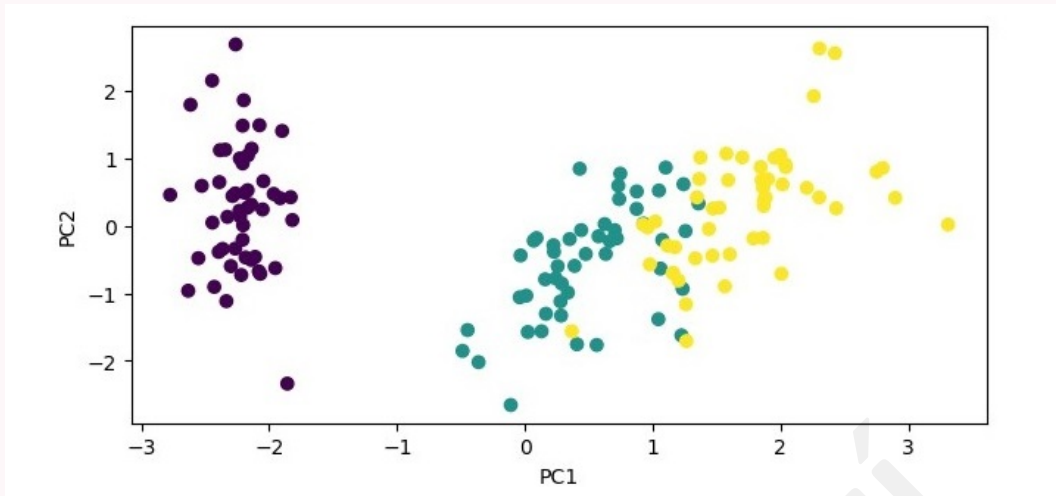
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y)
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```

Kết quả đầu ra

Explained variance ratio:

```
Explained variance ratio: [0.72962445 0.22850762]
```

Hai PC đầu giữ khoảng $72,9\% + 22,9\% \approx 95,8\%$ tổng phương sai (trên dữ liệu đã chuẩn hóa). Biểu đồ scatter PC1–PC2 tách ba loài iris rõ ràng.



9.3 Ưu và nhược điểm

Ưu điểm

- **Giảm chiều:** phù hợp dữ liệu nhiều feature, giữ phần lớn biến thiên.
- **Giảm tương quan:** PC mới trực giao, hỗ trợ mô hình nhảy đa cộng tuyến.
- **Dễ trực quan:** 2–3 PC đầu cho đồ thị 2D/3D.
- **Giảm overfitting:** ít chiều $\Rightarrow$ mô hình đơn giản hơn, tổng quát tốt hơn trên một số bài toán.
- **Tăng tốc tính toán:** ma trận đầu vào nhỏ hơn cho bước học sau.

Nhược điểm

- **Mất thông tin:** chiếu xuống $k < p$ không thể khôi phục hoàn toàn dữ liệu gốc.
- **Nhạy outlier:** một điểm cực đoan có thể kéo lệch PC.
- **Khó diễn giải:** PC là tổ hợp tuyến tính, không còn là “độ dài cánh hoa” cụ thể.
- **Giả định tuyến tính:** quan hệ phi tuyến mạnh cần kernel PCA hoặc phương pháp khác.
- **Bắt buộc chuẩn hóa:** bỏ qua bước này có thể làm PC theo đúng đơn vị lớn nhất thay vì cấu trúc thật.